

Penerapan Praktik DevOps Modul Intermediate Programmer

Pemetaan 12 bab modul *Training Programmer — DevOps v2* terhadap implementasi nyata di project Inventory & Order Management System, disertai nama file dan nomor baris sebagai bukti. Modul ini berpusat pada Kubernetes/Helm/observability stack, sementara brief IOMS secara eksplisit melarang Kubernetes, CI/CD, dan cloud deployment — sehingga sebagian besar bab dipetakan sebagai tidak relevan terhadap scope project ini, bukan kekurangan implementasi.

PROJECT

IOMS (ioms/)

DISUSUN OLEH

Arief Aryudi Syidik

REFERENSI MODUL

DevOps v2, Edisi 2.0

TANGGAL

2 Oktober 2026

- ✓ Diterapkan
- Sebagian
- ✕ Belum / Tidak relevan

BAB	TOPIK	STATUS
01	Production mindset dan operability contract	Sebagian
02	Metrics engineering dengan Prometheus	Tidak relevan
03	Grafana dashboards dan observability as code	Tidak relevan
04	Centralized logging yang aman dan dapat dicari	Sebagian
05	Distributed tracing dengan OpenTelemetry	Tidak relevan
06	Alerting, on-call, dan incident learning	Sebagian

BAB	TOPIK	STATUS
07	Docker image dan container lifecycle	Diterapkan
08	Production Dockerfile dan image optimization	Sebagian
09	Kubernetes architecture: Pod, Deployment, Service	Tidak relevan
10	Reliable workloads: probes, resources, rollout, self-healing	Tidak relevan
11	Helm, namespace, ServiceAccount, dan RBAC	Tidak relevan
12	Autoscaling, microservices, dan mini-project	Tidak relevan

01 Production Mindset dan Operability Contract

Sebagian

IOMS punya sebagian elemen operability (reproducibility, error handling berjenjang), tapi tidak punya service map, SLI/SLO, atau health-check endpoint formal — wajar untuk aplikasi internal skala kecil, tapi diakui sebagai gap terhadap konsep bab ini.

composer.lock (di-commit ke repo)

Dependency terkunci ke versi persis — build image reproducible di komputer mana pun, bukan "works on my machine".

app/Controller/Controller.php :75, 85, 96

Kegagalan ditangani berjenjang per jenis (403/422/500) dengan pesan aman ke user — bentuk dasar "failure mode" yang terdefinisi, meski belum berupa dokumen operability contract terpisah.

Catatan: tidak ada endpoint `/health`, service map, atau dokumen SLI/SLO — brief project tidak mewajibkan ini, dan skala IOMS (aplikasi internal, bukan layanan publik) belum membutuhkannya.

02 Metrics Engineering dengan Prometheus

Tidak relevan

Tidak ada endpoint metrics atau instrumentasi Prometheus. Brief project (§4.3, Di Luar Scope) tidak meminta observability stack seperti ini untuk IOMS.

03 Grafana Dashboards dan Observability as Code

Tidak relevan

Tidak ada dashboard Grafana. Dashboard yang ada di IOMS (`/dashboard`) adalah dashboard bisnis aplikasi (nilai inventori, order pending), bukan dashboard observability infrastruktur — beda domain dari yang dibahas bab ini.

04 Centralized Logging yang Aman dan Dapat Dicari

Sebagian

Ada logging dasar yang aman (tidak membocorkan detail teknis ke user), tapi belum berupa log terstruktur (JSON) atau tersentralisasi ke stack pencarian seperti ELK/Loki.

app/Controller/Controller.php :75, 85, 96 · **public/index.php** :180

`error_log()` mencatat pesan + konteks file:line untuk tiap exception, terpisah dari response yang dikirim ke user (`ini_set('display_errors', '0')` di `public/index.php:31`).

Catatan: log masih berupa baris teks bebas ke `stderr`/file Apache, bukan JSON terstruktur yang mudah di-query; tidak ada agregasi log lintas container (di luar scope brief, dan IOMS hanya 1 service aplikasi).

05 Distributed Tracing dengan OpenTelemetry

Tidak relevan

IOMS adalah monolith PHP native satu service (bukan microservices) — tidak ada request yang melintasi banyak service sehingga tidak ada kebutuhan distributed tracing. Sesuai brief §4.3 yang melarang arsitektur microservices sungguhan.

06 Alerting, On-Call, dan Incident Learning

Sebagian

Tidak ada alerting/on-call otomatis, tapi ada bentuk sederhana "incident learning" tertulis lewat pencatatan bug dan keterbatasan yang jujur.

docs/testing/known-bugs.md (34 baris)

Bug yang diketahui dicatat eksplisit dengan konteks, alih-alih disembunyikan — prinsip dasar yang sama dengan "incident learning": kegagalan didokumentasikan, bukan dihindari dari catatan.

docs/quality/tech-debt.md (25 baris)

10 keterbatasan sistem dicatat dengan alasan dan rencana perbaikan — bentuk paling dekat dengan "decision record" operasional yang dibahas modul, meski bukan post-mortem insiden produksi sungguhan.

Catatan: tidak ada alert rule, on-call rotation, atau runbook formal — di luar scope brief; script `scripts/check-low-stock.php` adalah satu-satunya bentuk "proactive check" yang ada, dan sifatnya bisnis (reorder point) bukan infrastruktur.

07 Docker Image dan Container Lifecycle

Diterapkan

Aplikasi dan database dapat dibangun serta dijalankan sepenuhnya lewat Docker Compose dari kondisi bersih, dengan urutan startup yang benar dan data seed otomatis.

docker-compose.yml :1-35

Dua service (app, mysql) dengan `depends_on: mysql: condition: service_healthy` — container aplikasi menunggu database benar-benar siap (bukan sekadar container running) sebelum start, mencegah race condition saat cold start.

docker-compose.yml :26-31

healthcheck memakai `mysqladmin ping` dengan interval/timeout/retries eksplisit — definisi "sehat" yang konkret, bukan asumsi container ada = siap dipakai.

docker-compose.yml :32-34

Volume `db_data` untuk persistensi antar-restart, dan `schema.sql/seed.sql` di-mount ke `docker-entrypoint-initdb.d/` agar database terisi otomatis hanya saat volume kosong (idempotent terhadap restart biasa).

Dockerfile :34-36

Layer `COPY composer.json composer.lock` dipisah sebelum `COPY . .` — memanfaatkan Docker layer cache: perubahan kode aplikasi tidak memaksa `composer install` dijalankan ulang.

Verifikasi langsung: `docker compose down -v && docker compose up -d --build` dari kondisi benar-benar kosong (volume dihapus) — container `mysql` healthy, container `app` start setelahnya, login & halaman `dashboard/produk` mengembalikan 200 dengan data seed lengkap, tanpa dependency eksternal apa pun di luar Docker Compose.

08 Production Dockerfile dan Image Optimization

Sebagian

Ada beberapa praktik optimasi image yang benar, tapi tiga praktik inti bab ini (multi-stage build, non-root runtime, `.dockerignore`) belum diterapkan — gap nyata yang diakui, bukan diklaim selesai.

Dockerfile :35

`composer install --no-dev --optimize-autoloader` — dependency development (PHPUnit, PHPStan, dst.) tidak ikut masuk ke image produksi; autoload class map dioptimasi, bukan classmap dinamis saat runtime.

Dockerfile :11

`rm -rf /var/lib/apt/lists/*` di baris RUN yang sama dengan `apt-get install` — mencegah cache APT ikut tersimpan sebagai layer terpisah yang memperbesar image final.

Gap 1 — Tidak ada `.dockerignore`: `COPY . .` (Dockerfile:39) menyalin seluruh isi direktori proyek ke dalam image, termasuk `.git/`, `tests/`, `docs/`, dan berpotensi file `.env` lokal jika ada saat build — membengkakkan ukuran image dan berisiko membocorkan credential ke dalam layer image jika build dilakukan sembarangan.

Gap 2 — Bukan multi-stage build: Dockerfile hanya punya satu FROM (baris 1) — tool build (Composer, git, unzip) ikut terbawa ke image runtime akhir, bukan dibuang di stage terpisah.

Gap 3 — Tidak ada instruksi USER non-root: tidak ada baris USER `www-data` eksplisit di Dockerfile; proses container secara default berjalan sebagai root sampai Apache men-drop privilege child process-nya sendiri secara internal — tidak sesuai prinsip least-privilege container yang dibahas modul.

09 Kubernetes Architecture: Pod, Deployment, Service

Tidak relevan

Brief project (§4.3, Di Luar Scope) secara eksplisit menyatakan Kubernetes tidak diwajibkan untuk IOMS. Orkestrasi cukup lewat Docker Compose dua service.

10 Reliable Workloads: Probes, Resources, Rollout, Self-Healing

Tidak relevan

Konsep readiness/liveness probe Kubernetes tidak berlaku tanpa Kubernetes. Padanan paling dekat di level Docker Compose adalah healthcheck pada service mysql (lihat Bab 07) dan `restart: unless-stopped` pada kedua service sebagai bentuk self-healing paling dasar (restart otomatis saat container crash).

11 Helm, Namespace, ServiceAccount, dan RBAC

Tidak relevan

Tanpa Kubernetes, tidak ada kebutuhan Helm chart atau RBAC cluster. Kontrol akses IOMS ditegakkan di level aplikasi (`Auth::requireRole()`) dan level database (user `ioms_user` non-root, bukan root, di `docker-compose.yml`:16–19) — sudah dibahas detail di `docs/report/PHP-Practice-Evidence.pdf` Bab 09.

12 Autoscaling, Microservices, dan Mini-Project

Tidak relevan

IOMS sengaja dibangun sebagai satu aplikasi monolith (brief melarang microservices sungguhan), sehingga autoscaling horizontal antar-service tidak relevan. Skala IOMS (aplikasi internal, data seed puluhan baris) tidak menuntut mekanisme scaling otomatis.

IOMS — Bukti Penerapan Praktik DevOps · Referensi: Training Programmer — DevOps v2 (PT Neuronworks Indonesia)